Lorenz System Neural ODE: Updated Implementation

About This Document

This note compares the **original implementation** (as documented in `Lorenz_System.pdf`) with the **updated implementation** in `lorenz.ipynb`, highlighting all key architectural and training changes.

System Overview

The Lorenz system remains unchanged — a 3-variable chaotic ODE:

$$\frac{dx}{dt} = \sigma(y - x)$$

$$\frac{dy}{dt} = x(\rho - z) - y$$

$$\frac{dz}{dt} = xy - \beta z$$

Where $\sigma = 10$, $\rho = 28$, $\beta = \frac{8}{3}$ (classical chaotic regime).

Key Changes: Old vs New

1. Neural Network Architecture

Aspect	Old	New
Activation	`SiLU`	`Tanh`
Weight Init	Xavier initialization	PyTorch default
Hidden Dim	`128` units	`128` units
Layers	`Linear(3,128) → SiLU → Linear(128,128) → SiLU → Linear(128,128) → SiLU → Linear(128,128) → SiLU → Linear(128,3)`	`Linear(3,128) → Tanh → Linear(128,128) → Tanh → Linear(128,3)`

⚠ **Activation Change**

- The old implementation used **SiLU** (Sigmoid Linear Unit), which is smoother and non-monotonic, generally better for chaotic systems.
- The new implementation uses **Tanh**, which is bounded and more classical for ODEs. This trade-off may affect gradient stability in long-horizon predictions.

---

2. Training Strategy

Major Change

Windowed Batching Replaced by DataLoader Sequences

Old approach (PDF):

```
# Random windowed sampling
start_idx = np.random.randint(0, len(t_stage1) - window_size)
t_batch = t_stage1[start_idx:start_idx + window_size]
y_batch = y_stage1[start_idx:start_idx + window_size]
y0_batch = y_batch[0]
```

New approach (Notebook):

```
# Sliding window sequences → DataLoader with batching
seq_length = 100
sequences = create_sequences(data_norm, seq_length)
train_loader = DataLoader(TensorDataset(sequences), batch_size=64,
shuffle=True)
```

What changed:

- Old used **random single windows** per epoch (stochastic, one window at a time)
- New uses **all overlapping sliding windows** compiled into a dataset, then batched with `batch_size=64`
- New adds `shuffle=True` on the DataLoader, providing randomization at the batch level

3. Data Normalization

> **New Addition**
>
> adds **Z-score normalization** before training, which was absent in the PDF implementation.

```
data_mean = data.mean(dim=0)
data_std  = data.std(dim=0)
data_norm = (data - data_mean) / data_std
```

And predictions are **denormalized** after inference:

```
pred_y = pred_y * data_std.cpu().numpy() + data_mean.cpu().numpy()
```

This stabilizes training by bringing all three variables (x, y, z) to comparable scales.

---

4. Optimizer & Regularization

Aspect	Old	New
L2 Regularization	No	Yes (`l2_lambda=1e-4`)
Early Stopping	No	Yes (`patience=30`)

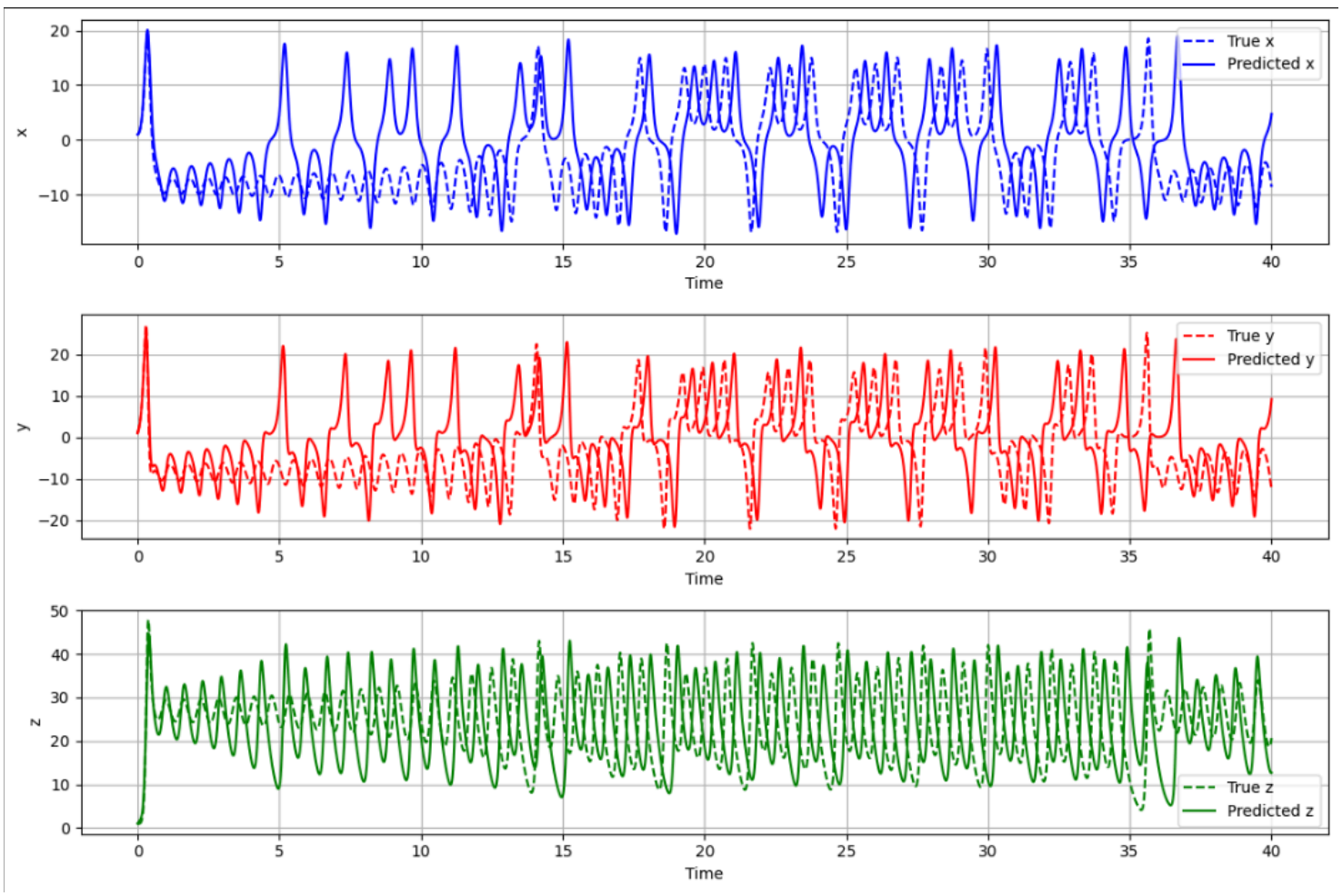
```
# L2 regularization in new notebook
l2_norm = sum(p.pow(2.0).sum() for p in func.parameters())
loss = loss + l2_lambda * l2_norm
```

Summary of All Changes

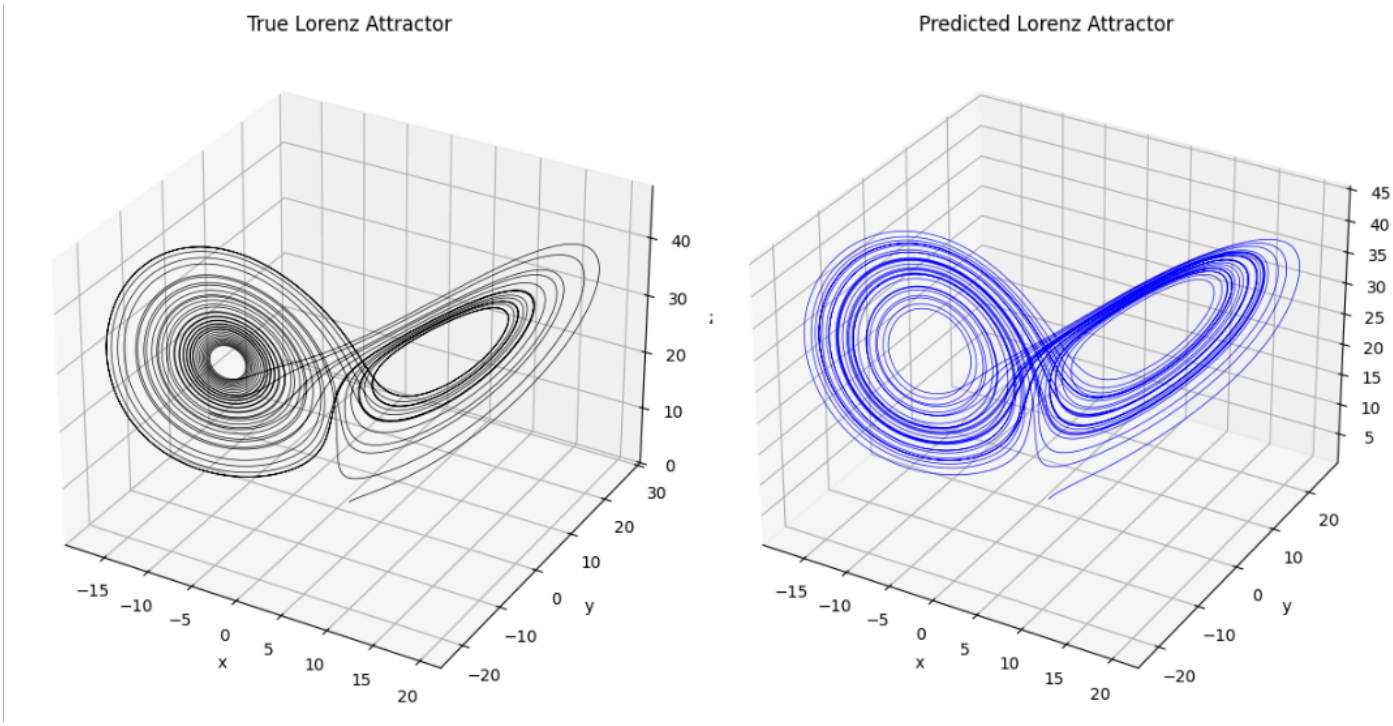
Old Notebook Implementation		New Notebook Implementation
SiLU activation	→	Tanh activation
Xavier init	→	Default init
Staged training (20/50/100%)	→	Single full-data training
Random windowed sampling	→	Sliding seq + DataLoader batch
No normalization	→	Z-score normalize + denormalize
LR Scheduler	→	Early stopping + L2 regularization

Outputs

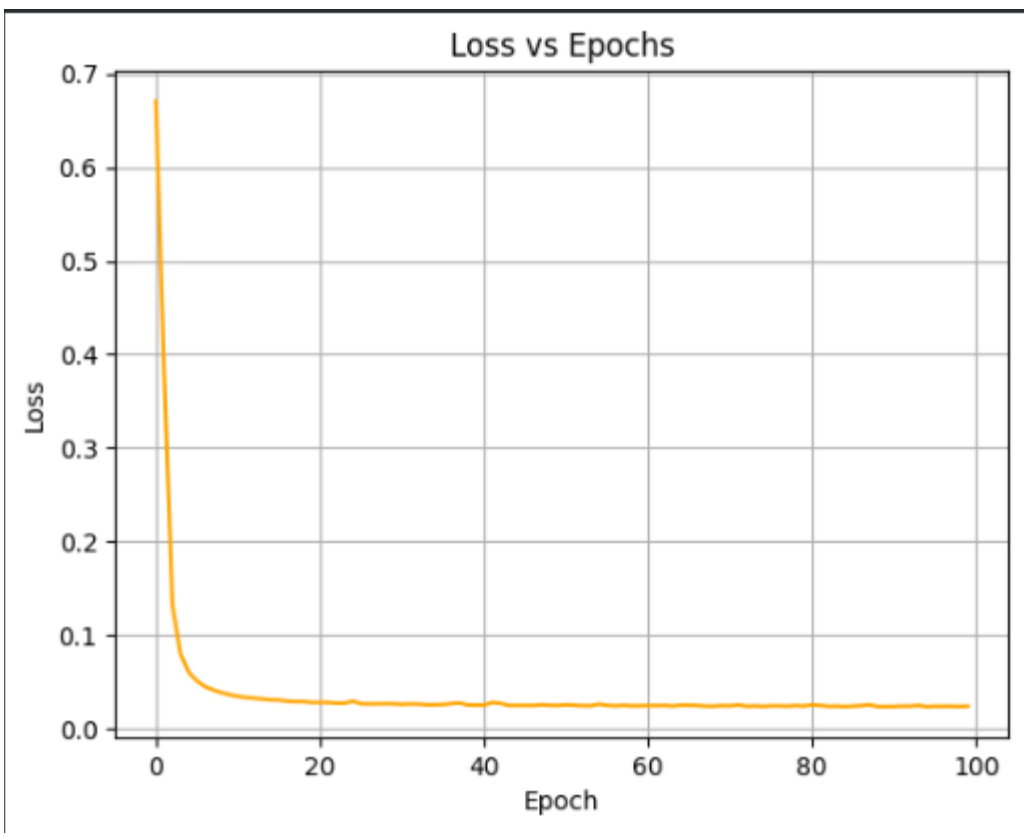
Time Series



3D Phase Plots



Loss Vs Epochs



Interpretation

✓ What Improved

- More standard, reproducible training pipeline (DataLoader, batching)
- Normalization improves gradient stability
- L2 regularization reduces overfitting
- Loss coverges unlike windowed learning approach

References

- Old implementation documented in: `Lorenz_System.pdf`
- New implementation: `lorenz.ipynb`